

Snake Game on ATmega32

Project Proposal

Embedded Systems Course | Microcontroller Project

Team 6: Malak Mamdouh, Fatma Eslam, Reem Eion, Merna Mohamed

1. Introduction / Motivation

The objective of this project is to implement the classic Snake game as an embedded system using the ATmega32 microcontroller. The project builds on the same core skills used in earlier lab exercises (sensor/input handling, timed control loops, and driving an output display) but shifts the emphasis toward game logic, real-time timing, and matrix/graphics display control. The result is a fully self-contained, playable hardware game with no PC dependency after programming.

2. Objectives

- Design and simulate a working Snake game circuit around the ATmega32 in Proteus.
- Develop firmware in Eclipse (AVR-GCC) implementing game logic: snake movement, growth, collision detection, and scoring.
- Display the game on an output device LCD.
- Accept directional input from push buttons (up/down/left/right).
- Implement game-over detection (wall collision, self-collision) and score display/reset.
- Add sound effects using buzzers in case of collision and earning points.

3. System Overview

The system consists of three main stages: Input, Processing, and Output, as summarized below.

Stage	Component	Function
Input	4x Push buttons (UP/DOWN/LEFT/RIGHT)	Read player direction commands via digital I/O pins with internal pull-ups
Processing	ATmega32 Microcontroller	Runs game loop: updates snake position, checks collisions, updates score
Output	16× 2 LCD screen	Displays snake, food, and game boundaries and “Game Over” message
Output	2x 7 segments	Displays score
Output	Buzzer	Audio feedback on eating food / game over

4. Simulation & Testing Plan (Proteus)

- Build incrementally: first confirm LED matrix driver displays a single static pixel/pattern correctly.
- Add push-button input handling and verify direction changes reflect correctly (test using virtual switches in Proteus).
- Implement and test the timer-based game tick using Proteus's built-in oscilloscope/graph tools to confirm consistent timing.
- Add snake growth and food-spawn logic, verify collision detection triggers Game Over correctly.
- Use a Virtual Terminal (UART debug prints) to monitor internal game-state variables during simulation for debugging.